



CAMBRIDGE

Lecture 5: Representation Learning

Stefan Bucher

MACHINE LEARNING IN ECONOMICS
UNIVERSITY OF CAMBRIDGE

Stefan Bucher

 Open in Colab

Prince (2023, chaps. 10, 12).¹

- . Figures taken or adapted from Prince (2023). All rights belong to the original author and publisher. These materials are intended solely for educational purposes.

Computer Vision and Convolutional Neural Networks

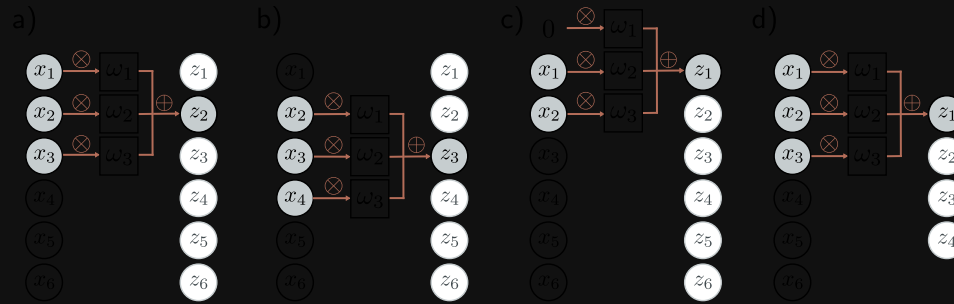
Prince (2023, chap. 10)

Convolutional Neural Networks (CNN)

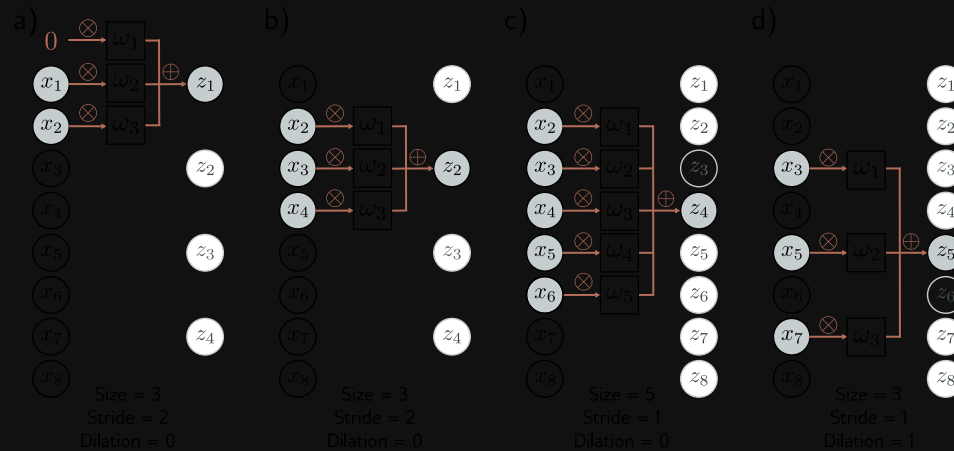
CNNs deal with

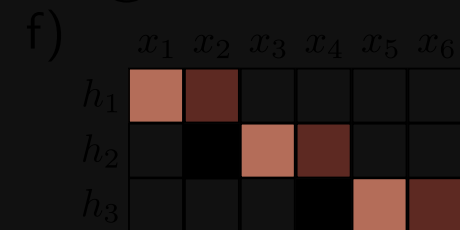
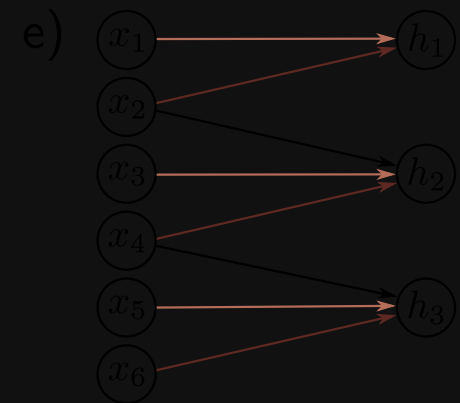
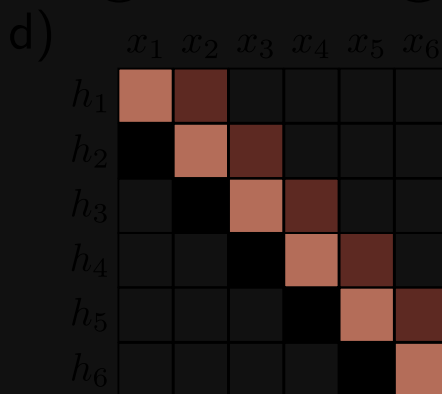
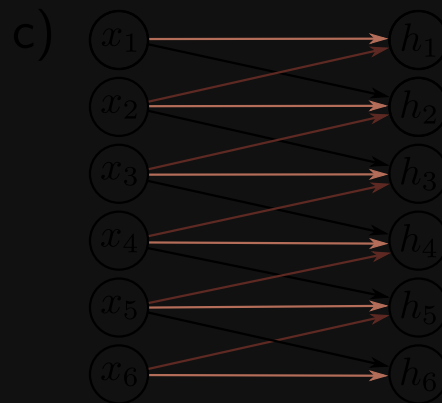
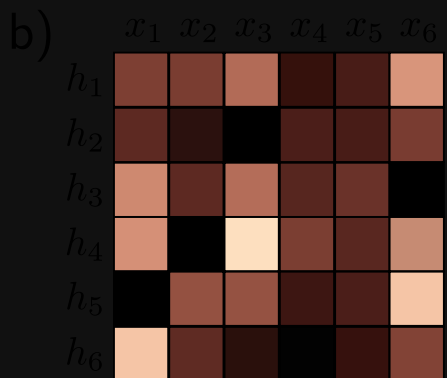
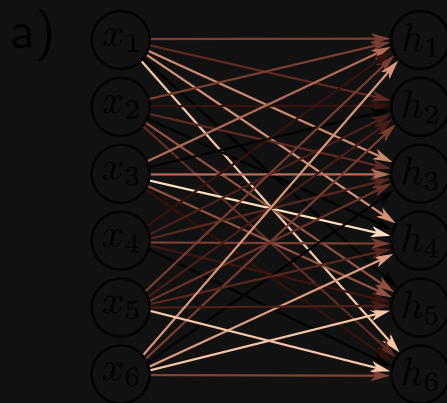
- high-dimensional images preventing fully connected networks
- local spatial statistics
- invariance ($f(t(x)) = f(x)$) and equivariance ($f(t(x)) = t(f(x))$) to transformations t

Convolution (1D)



Convolution kernel/filter amounts to *shared weights*.



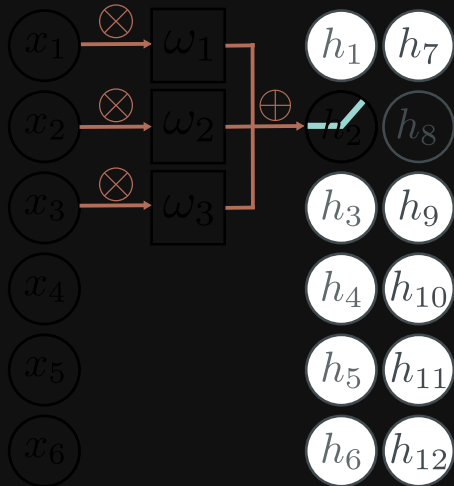


$$h_i = a \left(\beta + \sum_{j=1}^3 w_j x_{i+j-2} \right)$$

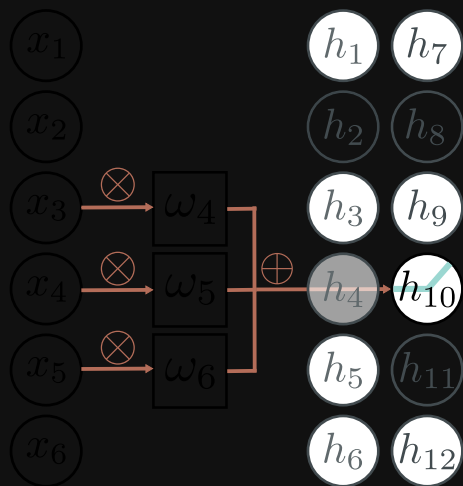
shares weights across i .

Channels / Feature Maps

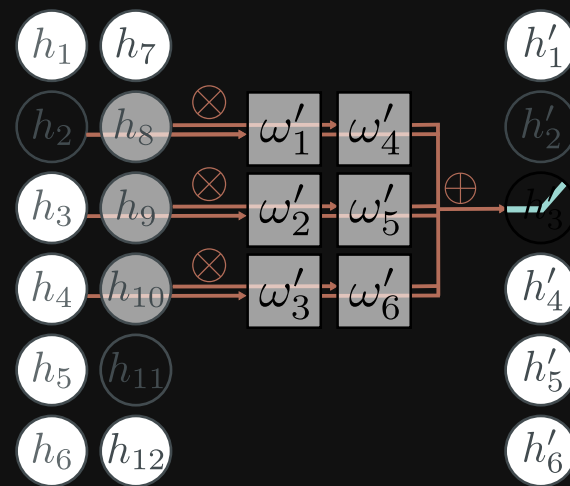
a)



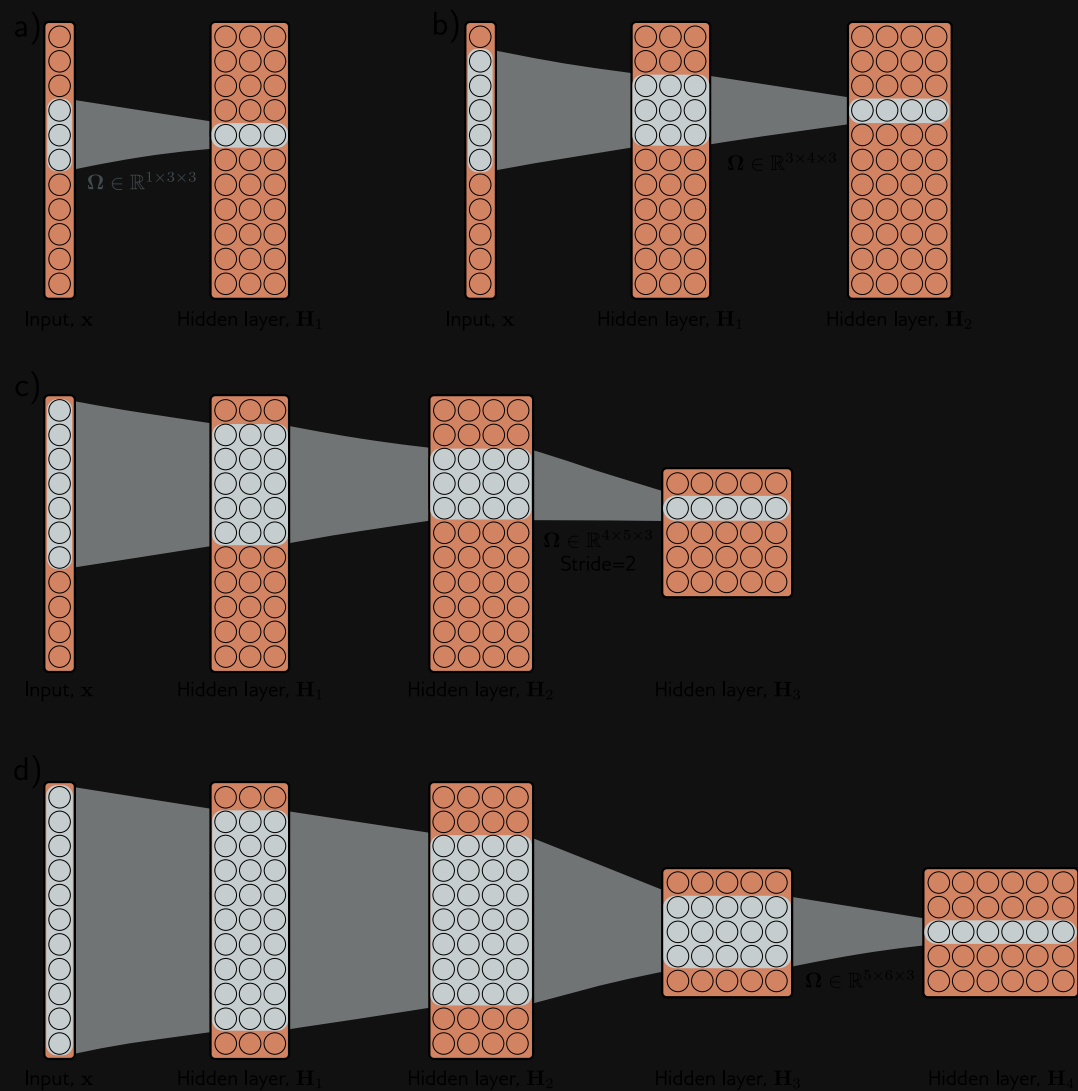
b)



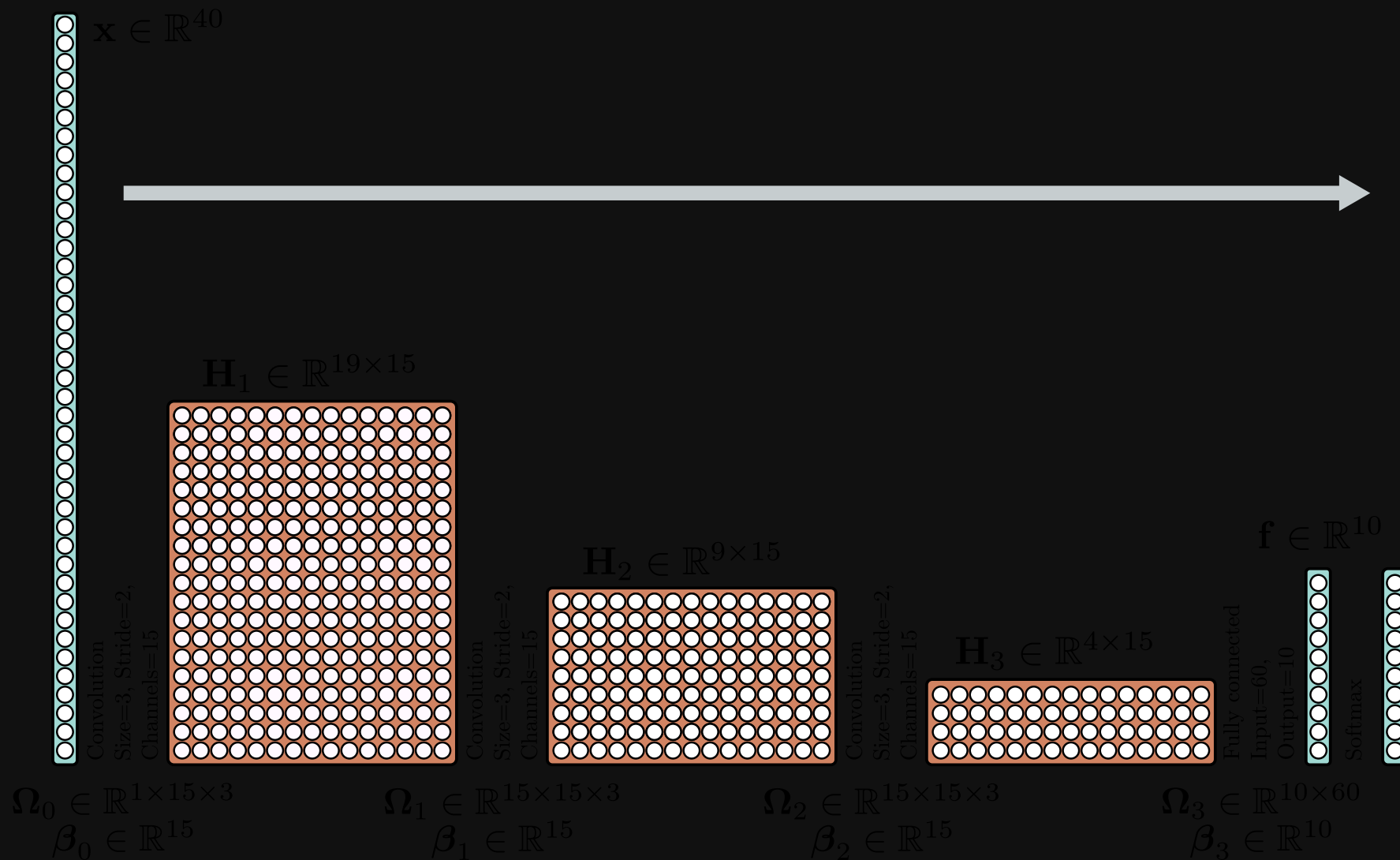
c)



Receptive Fields

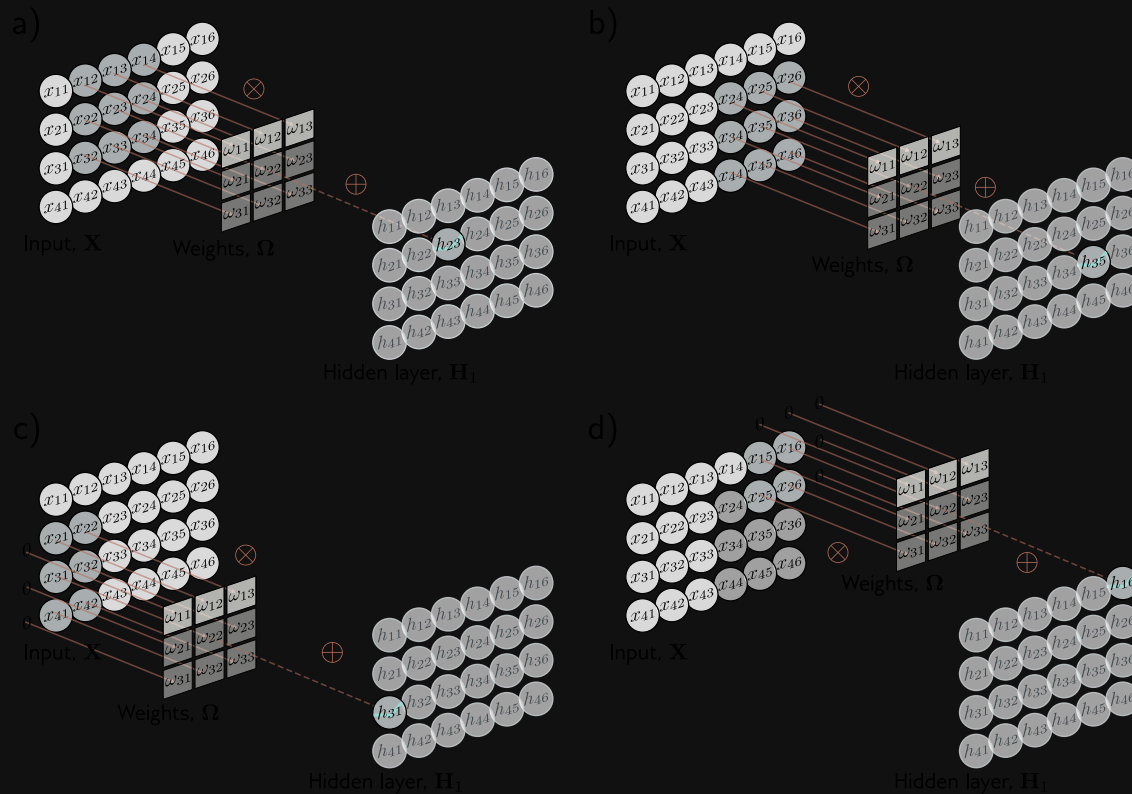


Example: MNIST-1D

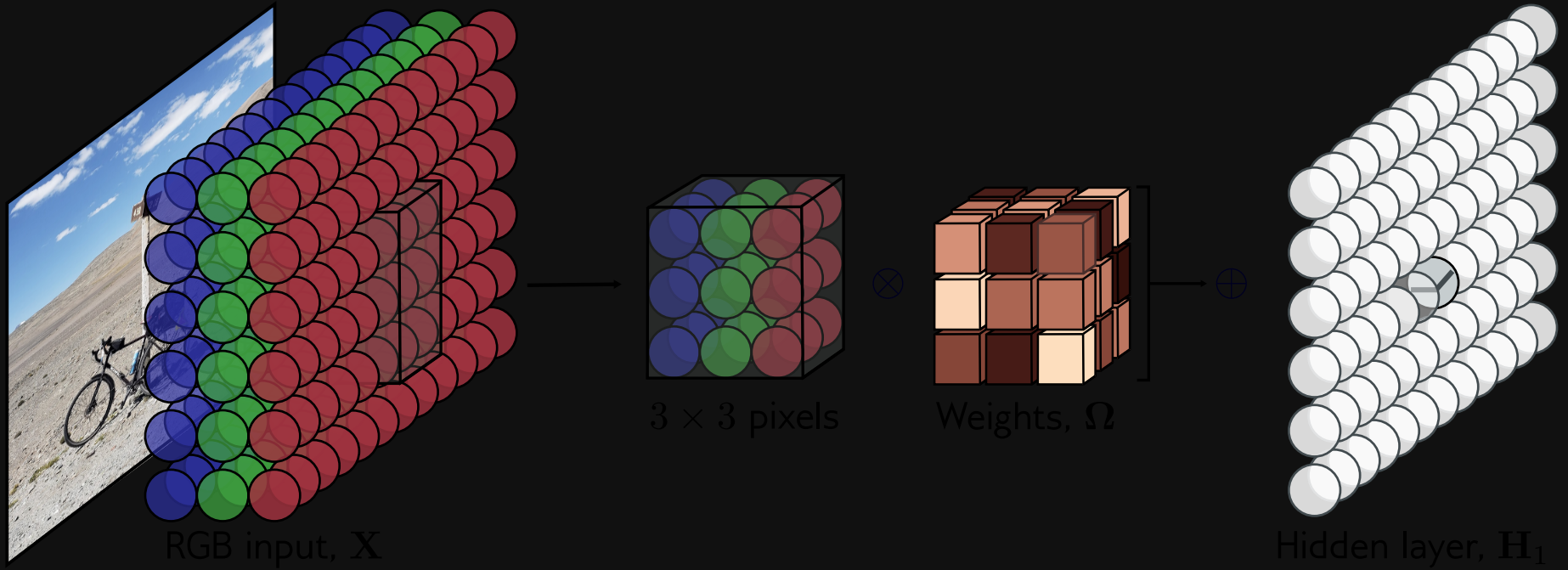


Convolutional Networks (2D)

$$h_{ij} = a \left(\beta + \sum_{m=1}^3 \sum_{n=1}^3 w_{mn} x_{i+m-2, j+n-2} \right)$$

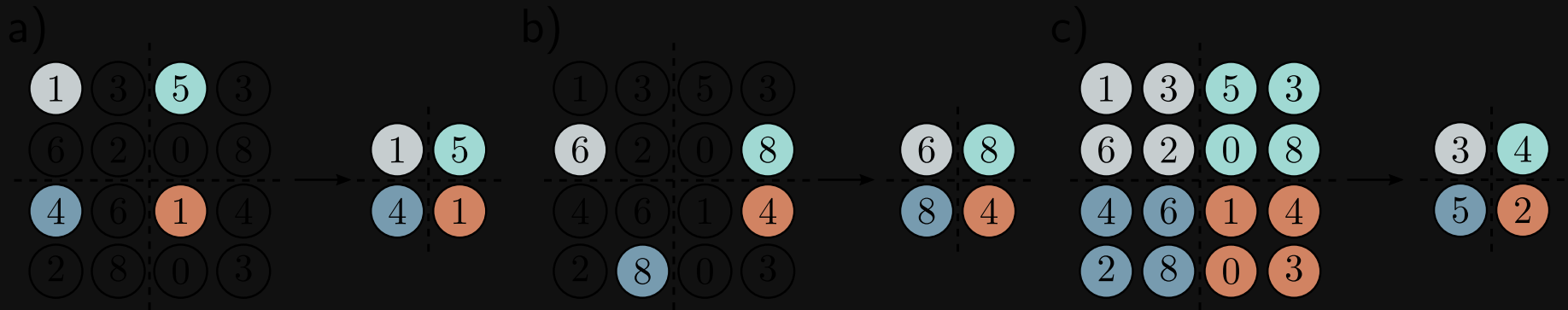


Example



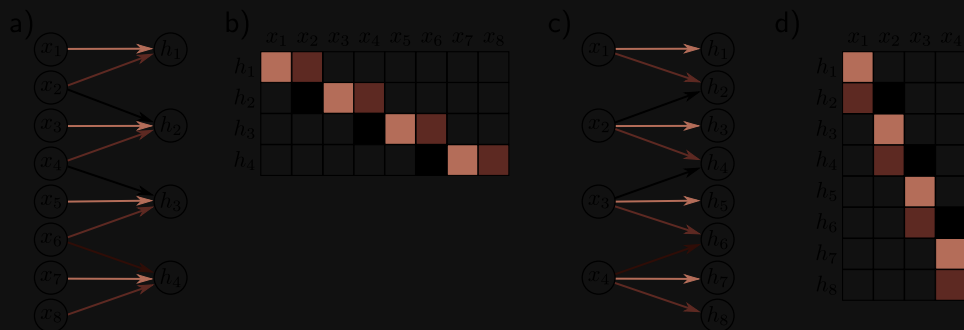
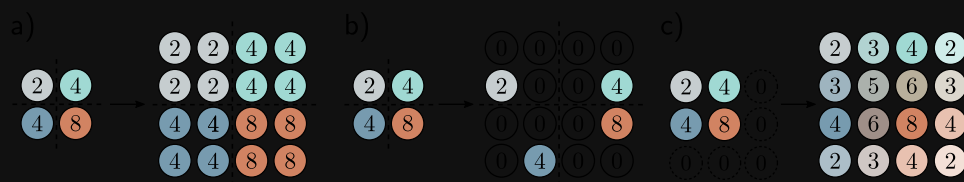
Downsampling

- Stride > 1
- Max Pooling
- Average Pooling

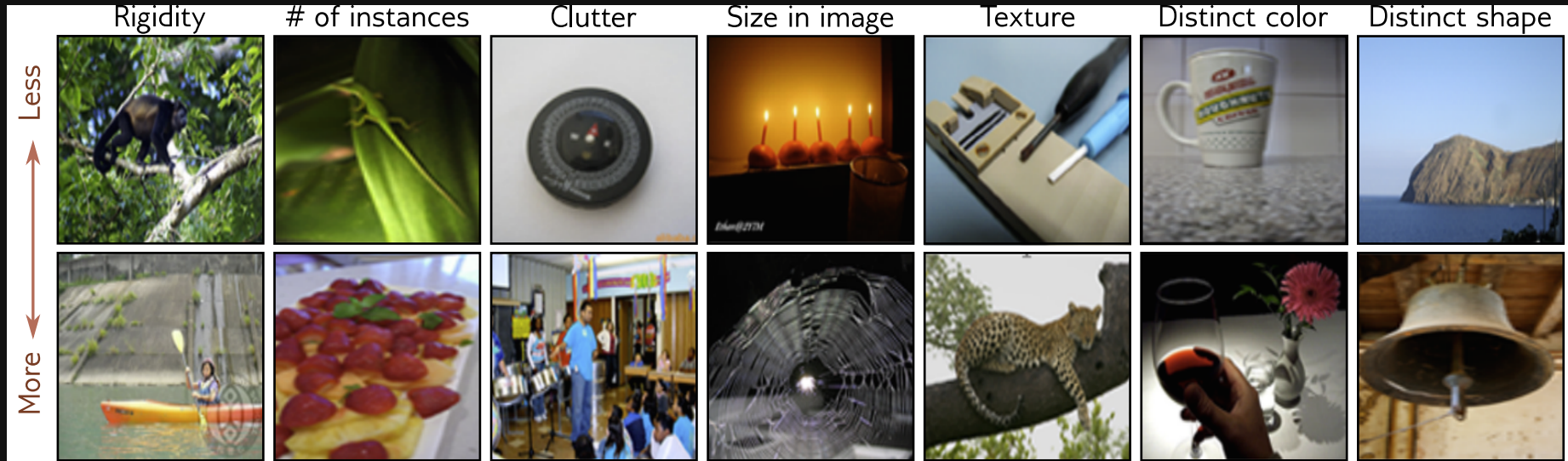


Upsampling

- Replicate
- Max Unpooling
- Bilinear Interpolation
- Transposed Convolution

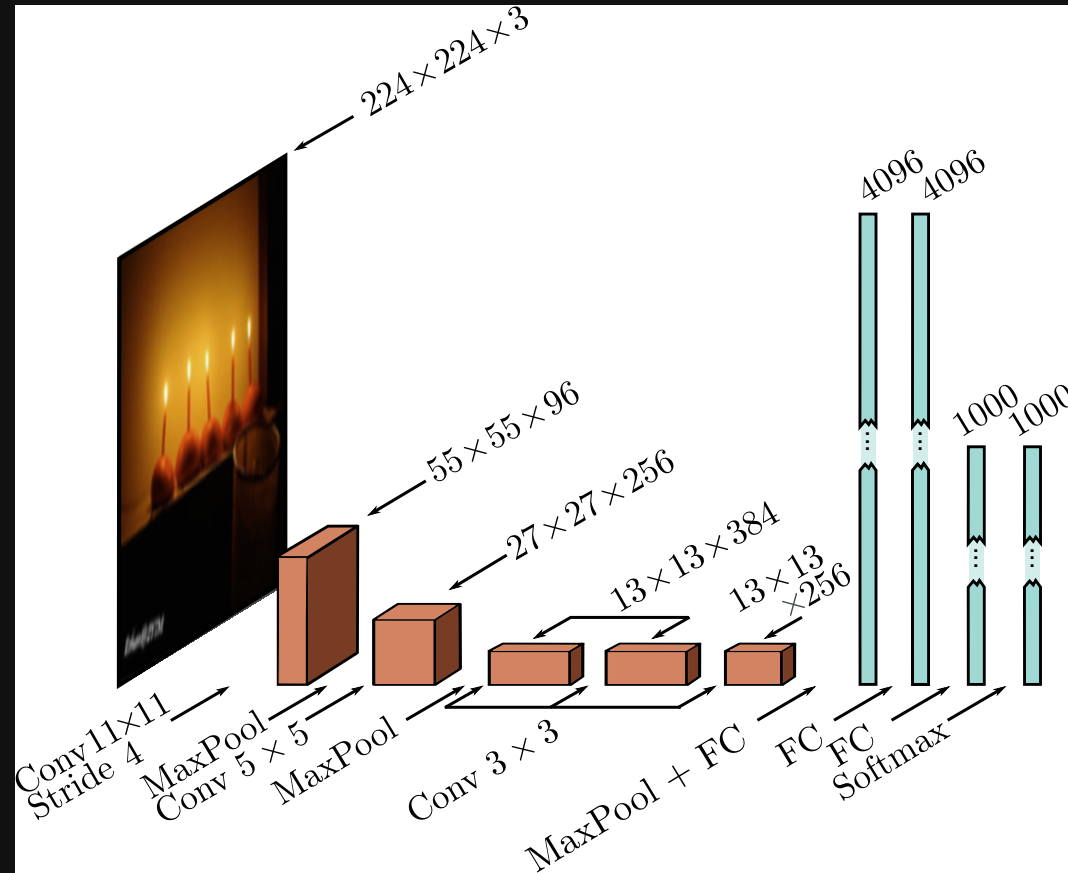


Application: Image Classification



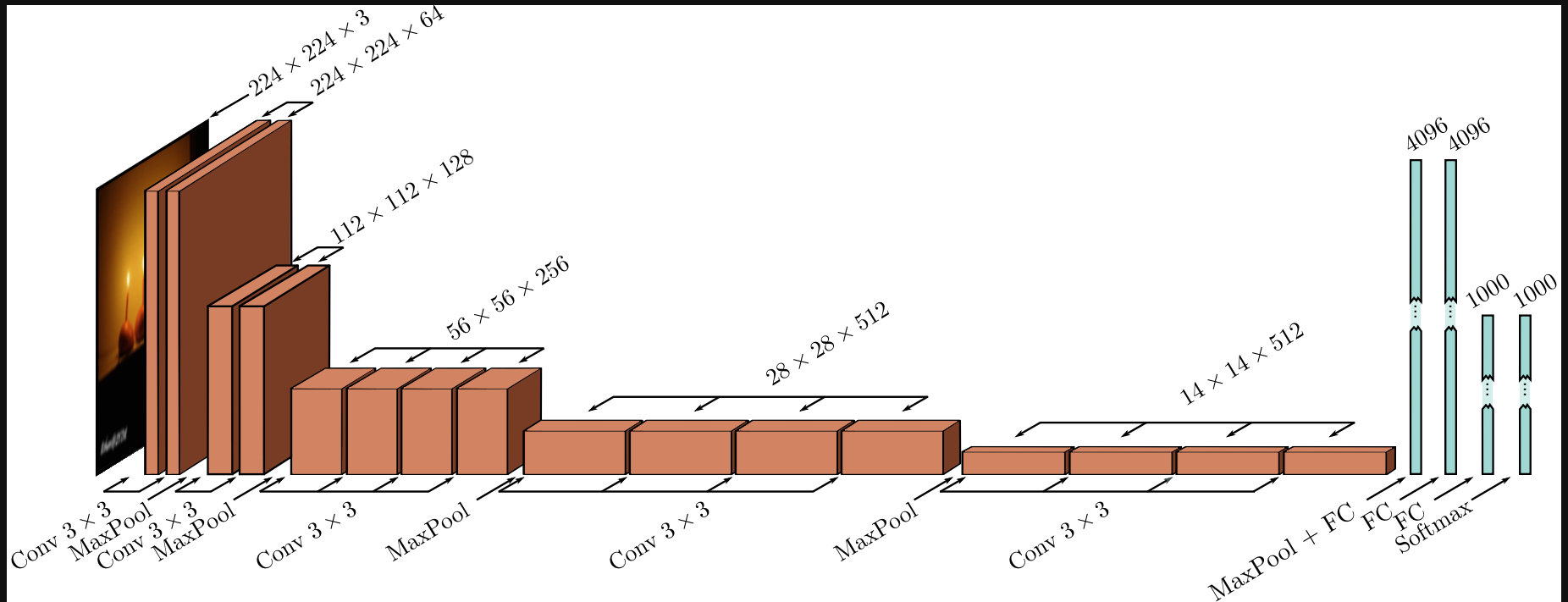
ImageNet: 1.2 million images, 1000 classes

AlexNet (2012)



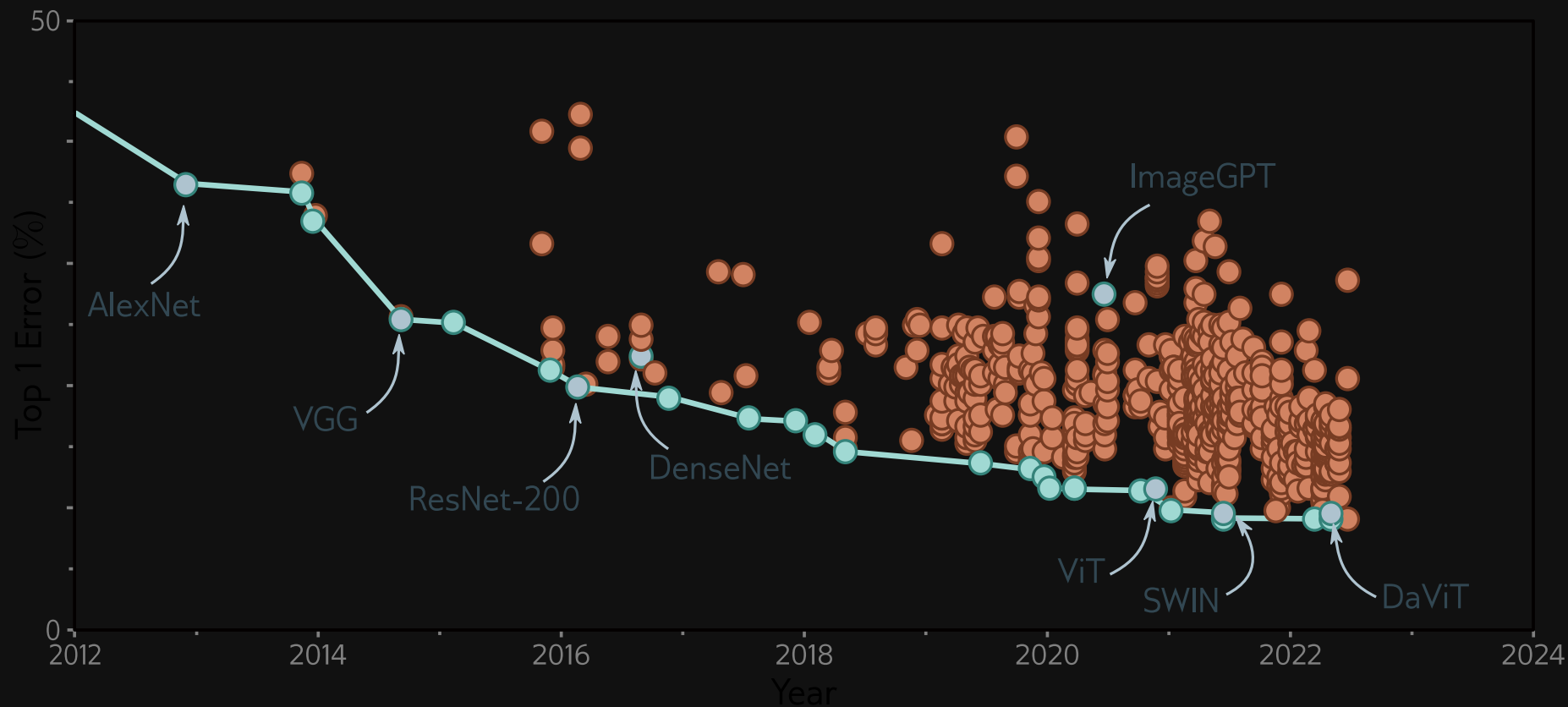
- 60 million parameters, 8 hidden layers
- 38.1% top-1 error, 16.4% top-5 error rate

VGG (2014)

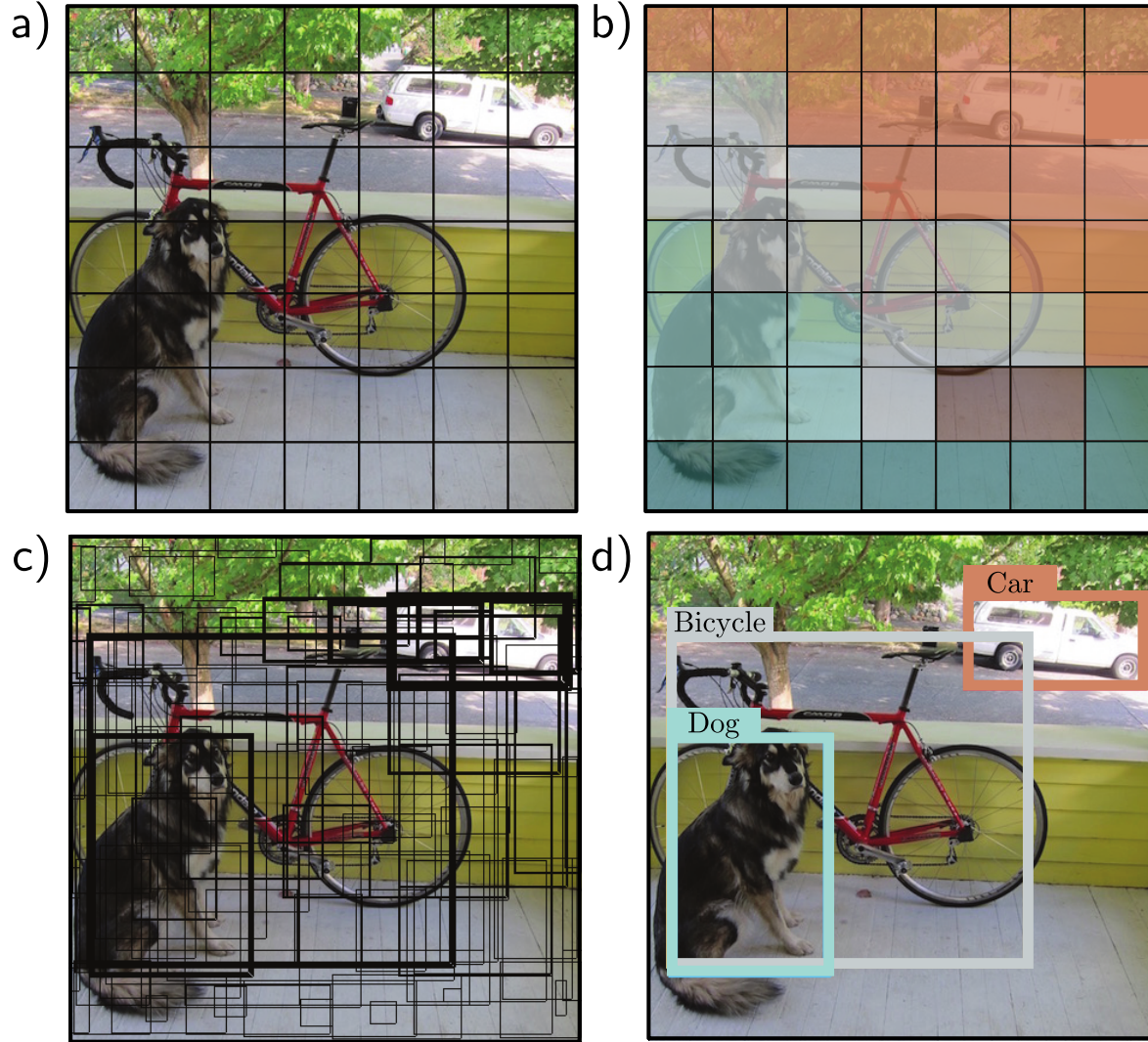


- 144 million parameters, 19 hidden layers
- 23.7% top-1 error, 6.8% top-5 error rate

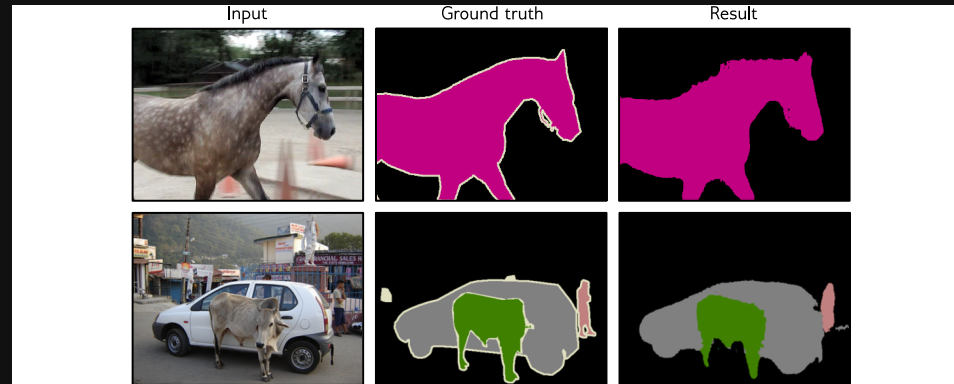
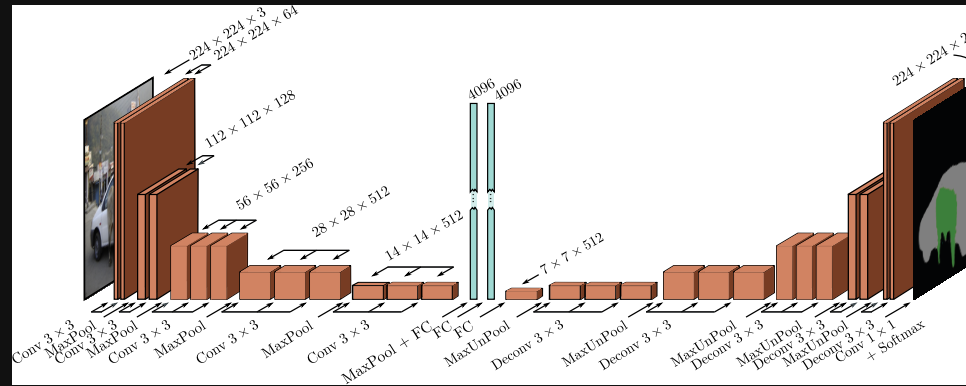
ImageNet Performance



Application: Object Detection (YOLO)



Application: Semantic Segmentation



Natural Language Processing and Transformers

Prince (2023, chap. 12)

NLP Pipeline: Subword Tokenization

a) a_sailor_went_to_see_see_see_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	s	a	t	o	h	i	u	b	d	w	c	f	i	m	n	p	r
33	28	15	12	11	8	6	6	4	3	3	3	2	1	1	1	1	1	1

b) a_sailor_went_to_see_see_see_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

_	e	se	a	t	o	h	i	u	b	d	w	c	s	f	i	m	n	p	r
33	15	13	12	11	8	6	6	4	3	3	3	2	2	1	1	1	1	1	1

c) a_sailor_went_to_see_see_see_
to_see_what_he_could_see_see_see_
but_all_that_he_could_see_see_see_
was_the_bottom_of_the_deep_blue_sea_sea_sea_

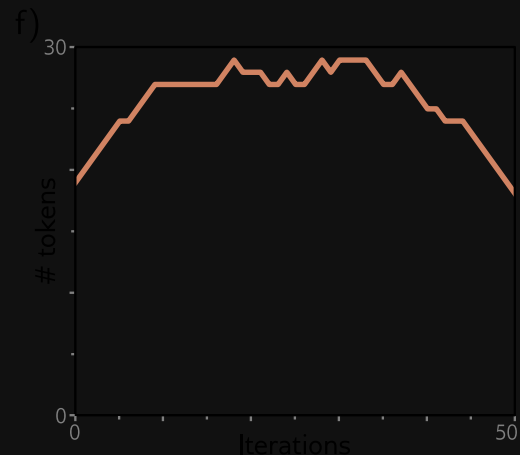
_	se	a	e	t	o	h	i	u	b	d	e	w	c	s	f	i	m	n	p	r
21	13	12	12	11	8	6	6	4	3	3	3	3	2	2	1	1	1	1	1	1

⋮ ⋮

d) see_sea_e b l w a could_hat he o t t the to u a d f m n p s sailor to
7 6 4 3 3 3 3 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1

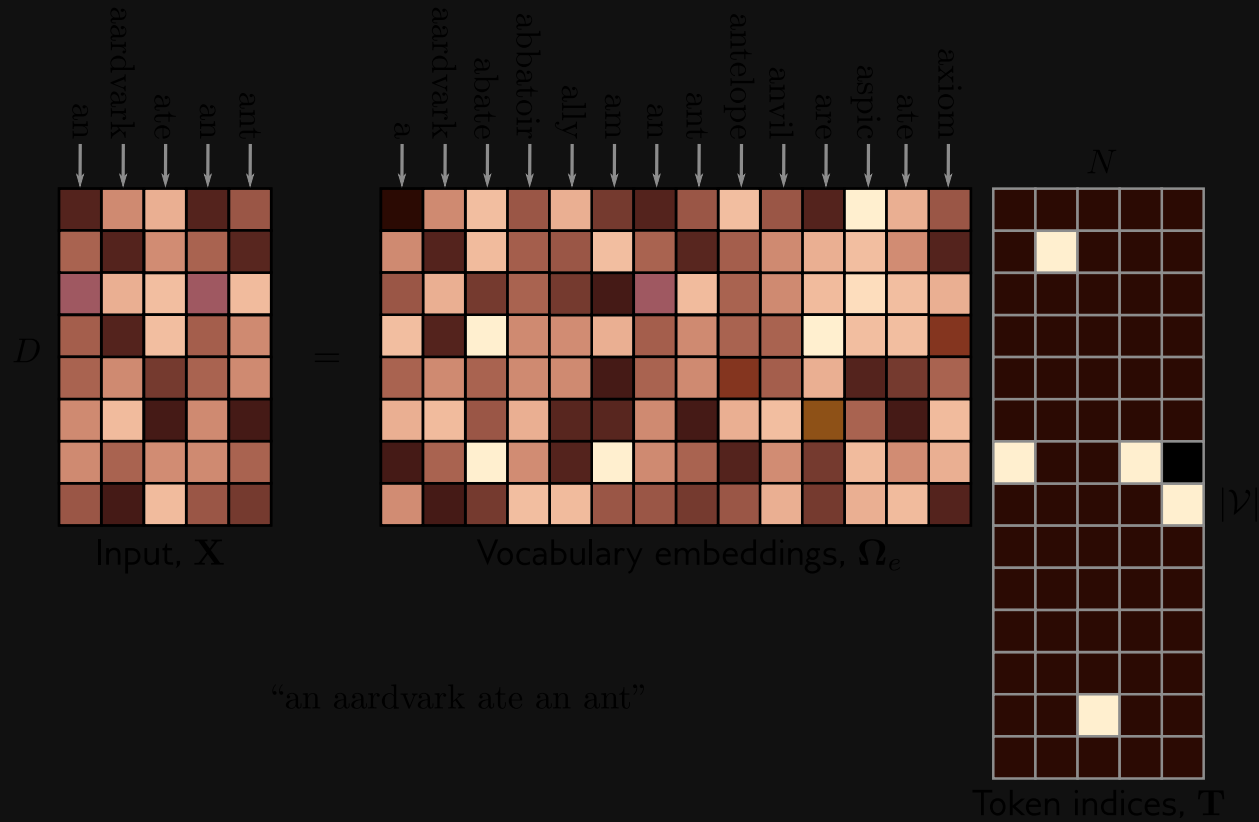
⋮ ⋮ ⋮

e) see_sea_could_he_the_a_all_blue_bottom_but_deep_of_sailor_that_to_was_went_what_
7 6 2 2 2 1



Byte Pair Encoding (BPE) greedily merges commonly occurring sub-strings based on frequency.

NLP Pipeline: Embedding



"an aardvark ate an ant"

Typically $\Omega_e \in \mathbb{R}^{1024 \times 30000}$, which is also learned.

Dot-Product Self-Attention

$$\mathbf{v}_m = \beta_v + \mathbf{\Omega}_v \mathbf{x}_m \quad m = 1, \dots, N$$

$$\mathbf{q}_n = \beta_q + \mathbf{\Omega}_q \mathbf{x}_n \quad n = 1, \dots, N$$

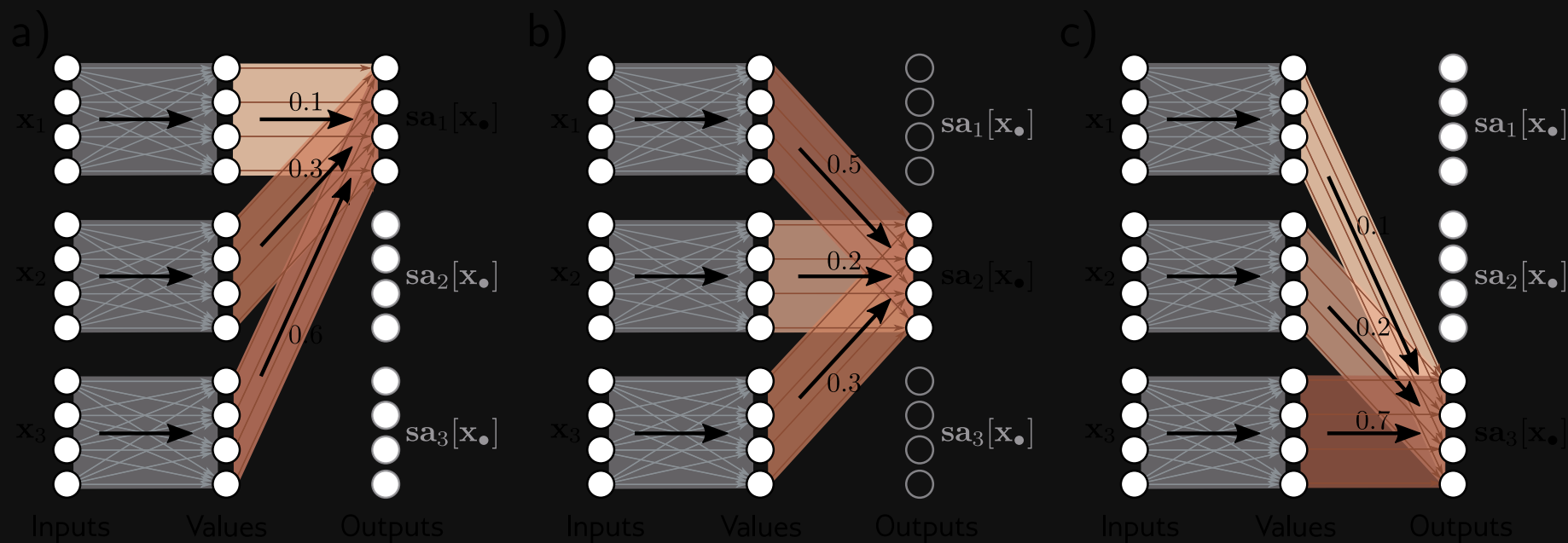
$$\mathbf{k}_m = \beta_k + \mathbf{\Omega}_k \mathbf{x}_m \quad m = 1, \dots, N$$

$$a[\mathbf{x}_m, \mathbf{x}_n] = \frac{\exp(\mathbf{k}_m^T \mathbf{q}_n)}{\sum_{m'=1}^N \exp(\mathbf{k}_{m'}^T \mathbf{q}_n)} \quad m, n = 1, \dots, N$$

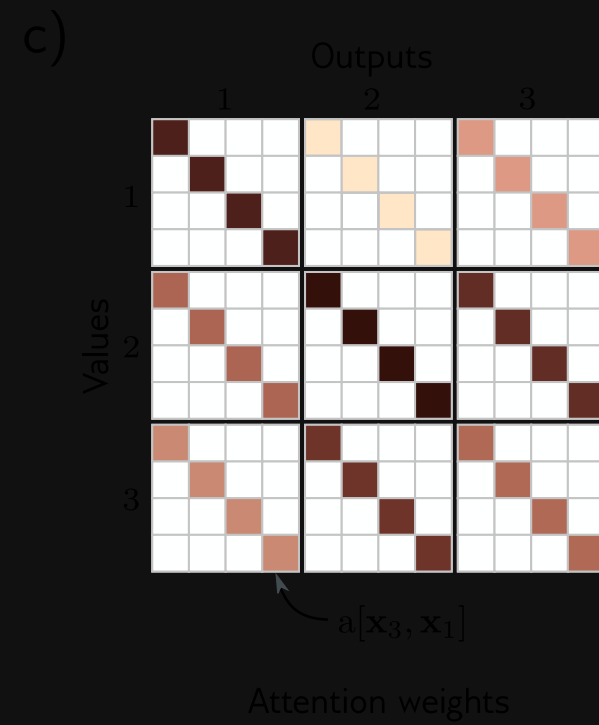
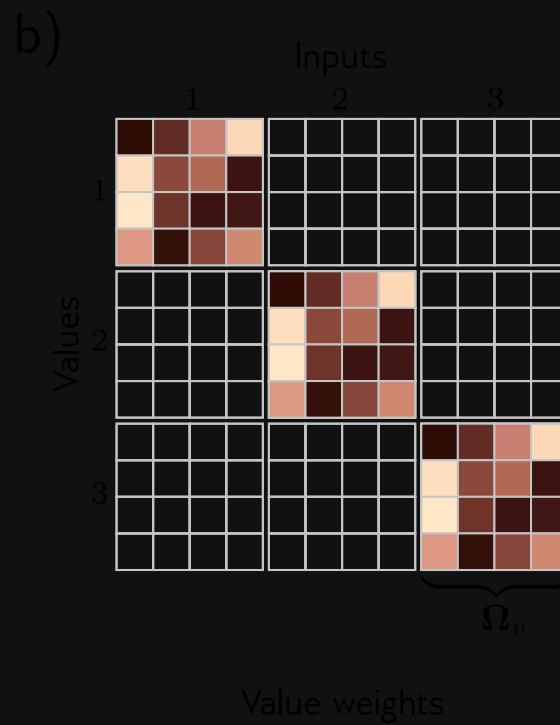
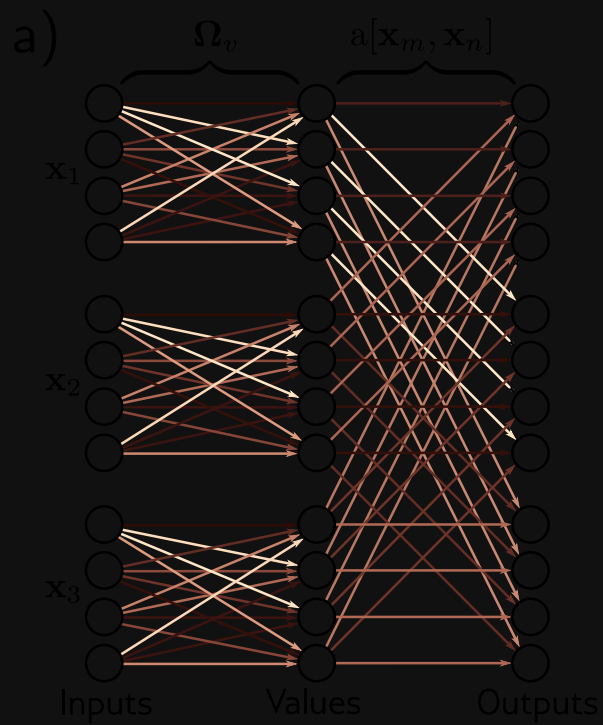
$$\mathbf{sa}_n[\mathbf{x}_1, \dots, \mathbf{x}_N] = \sum_{m=1}^N a[\mathbf{x}_m, \mathbf{x}_n] \mathbf{v}_m \quad n = 1, \dots, N$$

- inputs $\mathbf{x}_m \in \mathbb{R}^D$, outputs $\mathbf{sa}_n \in \mathbb{R}^D$
- values $\mathbf{v}_m \in \mathbb{R}^D$, queries $\mathbf{q}_n \in \mathbb{R}^{D_q}$, keys $\mathbf{k}_m \in \mathbb{R}^{D_q}$
- keys $\mathbf{k}_m$ compete for *attention* $a[\mathbf{x}_m, \mathbf{x}_n]$ the n -th output pays to input/value $\mathbf{v}_m$ (which depends on their similarity to query $\mathbf{q}_n$)

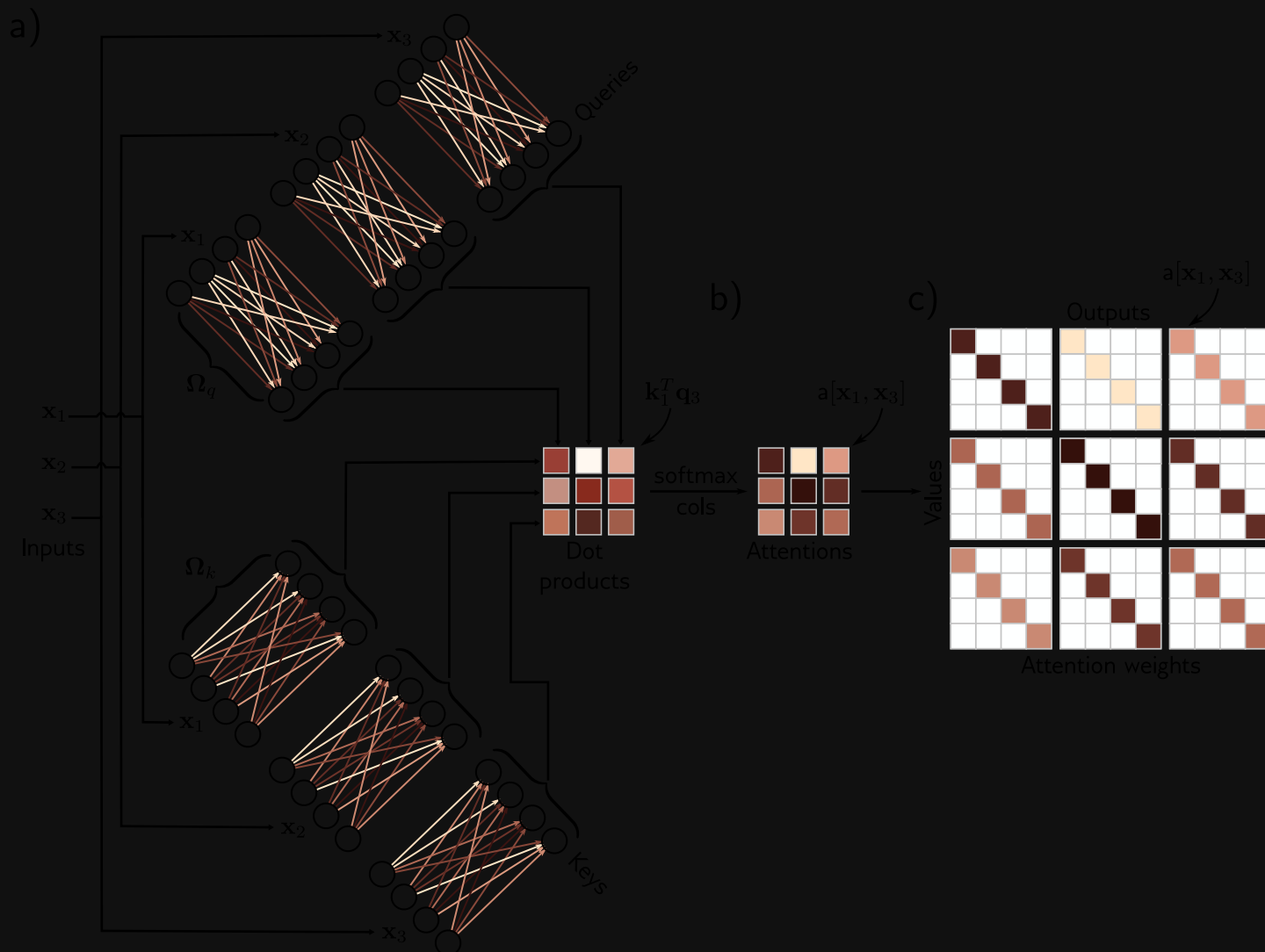
Self-Attention as Routing



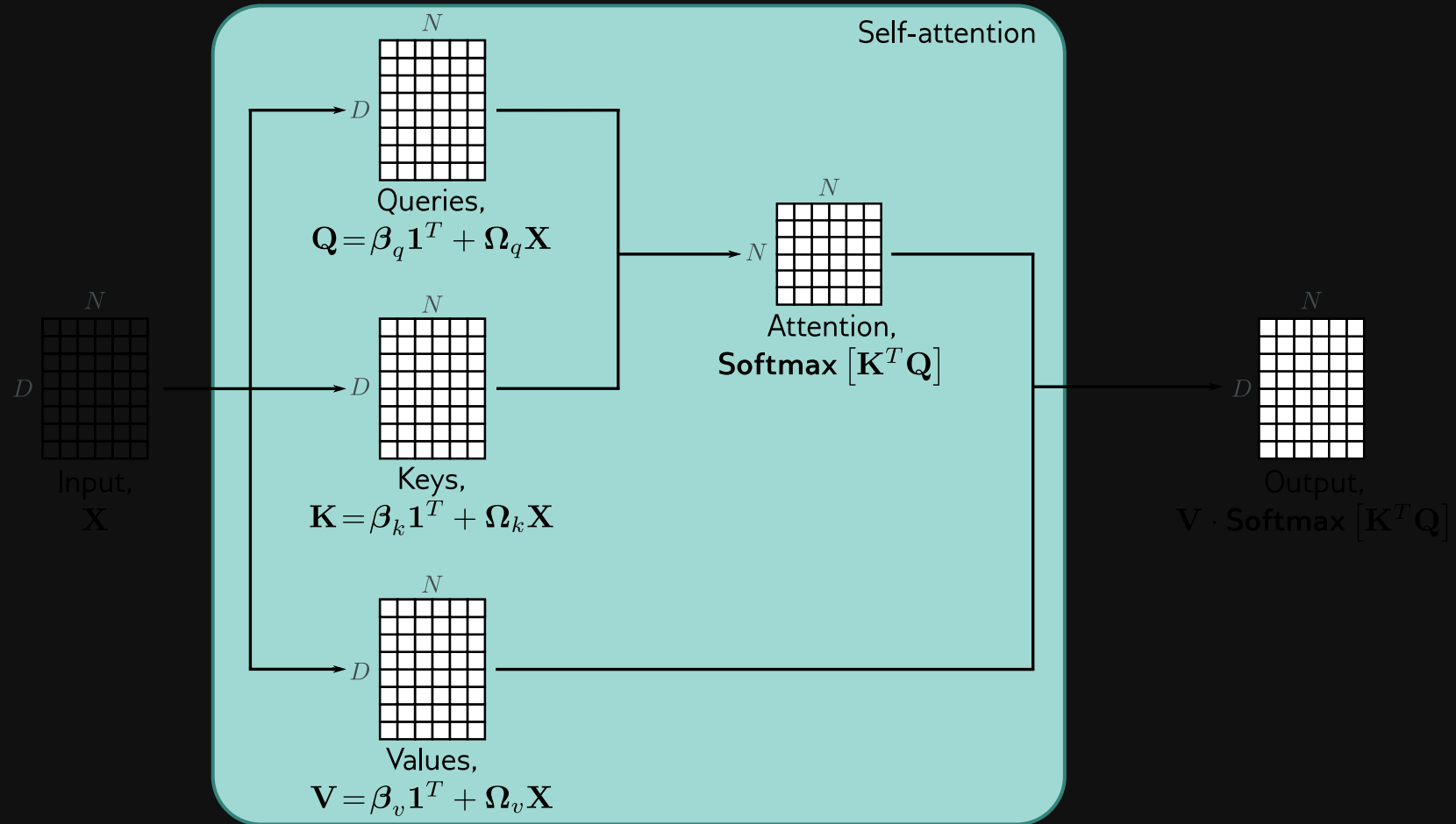
Self-Attention



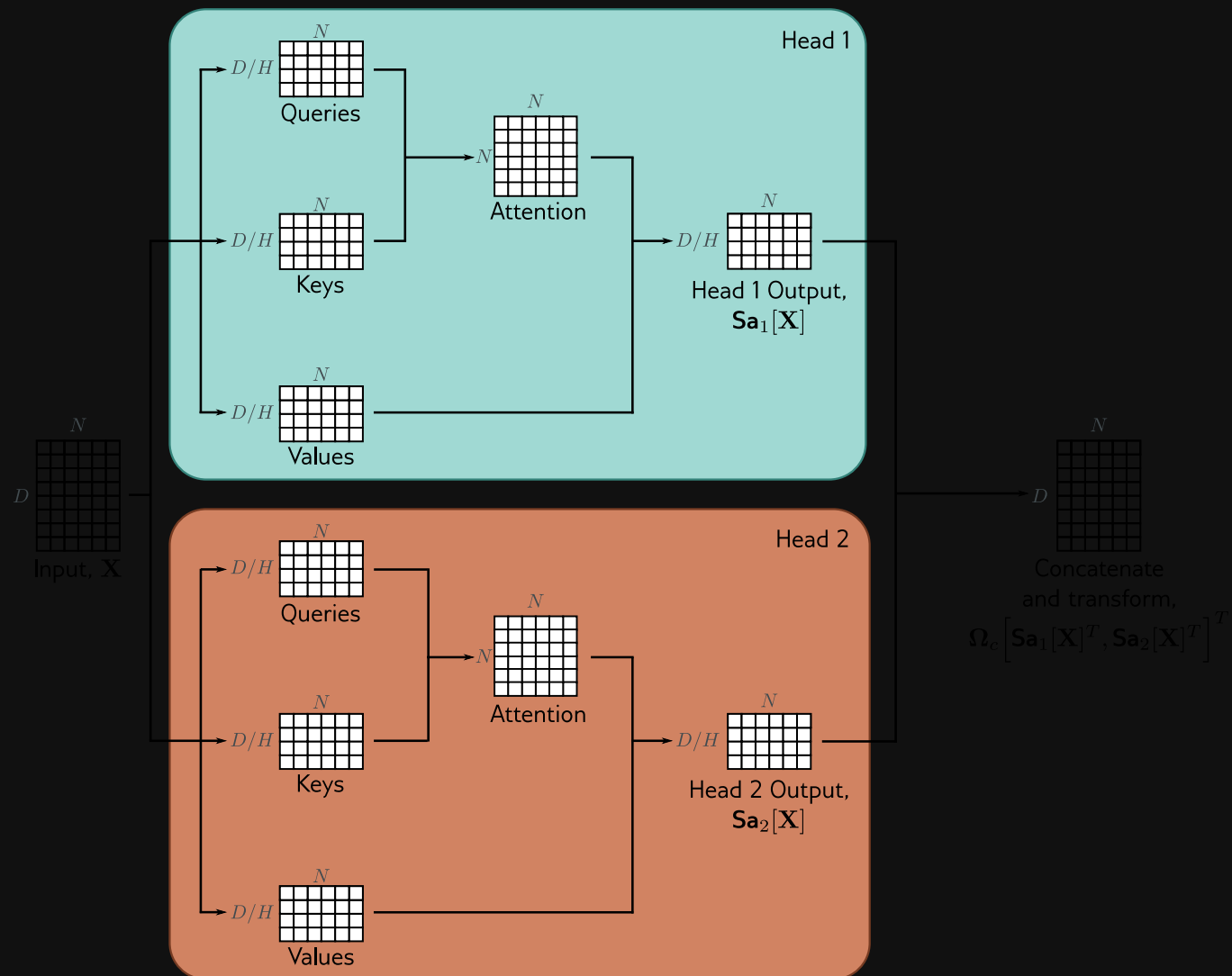
Computing Attention Weights



Dot-Product Self-Attention in Matrix Form



Multi-Head Self-Attention



Multi-Head Self-Attention (+scaling)

$$\mathbf{V}_h = \beta_{vh} \mathbf{1}^T + \mathbf{\Omega}_{vh} \mathbf{X}$$

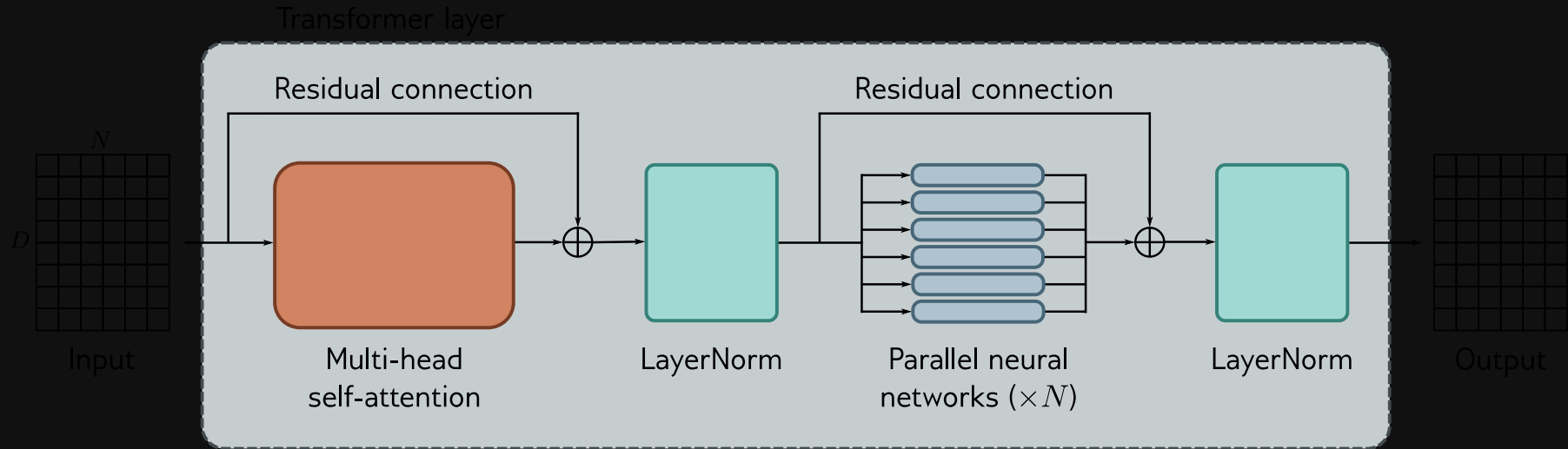
$$\mathbf{Q}_h = \beta_{qh} \mathbf{1}^T + \mathbf{\Omega}_{qh} \mathbf{X}$$

$$\mathbf{K}_h = \beta_{kh} \mathbf{1}^T + \mathbf{\Omega}_{kh} \mathbf{X}$$

$$\mathbf{Sa}_h[\mathbf{X}] = \mathbf{V}_h \mathbf{Softmax} \left[\frac{\mathbf{K}_h^T \mathbf{Q}_h}{\sqrt{D_q}} \right]$$

$$\mathbf{MhSa}[\mathbf{X}] = \mathbf{\Omega}_c [\mathbf{Sa}_1[\mathbf{X}]^T, \dots, \mathbf{Sa}_H[\mathbf{X}]^T]^T$$

Transformer Layer



$$\mathbf{X} \leftarrow \mathbf{X} + \text{MhSa}[\mathbf{X}]$$

$$\mathbf{X} \leftarrow \text{LayerNorm}[\mathbf{X}]$$

$$\mathbf{x}_n \leftarrow \mathbf{x}_n + \text{mlp}[\mathbf{x}_n] \quad n = 1, \dots, N$$

$$\mathbf{X} \leftarrow \text{LayerNorm}[\mathbf{X}]$$

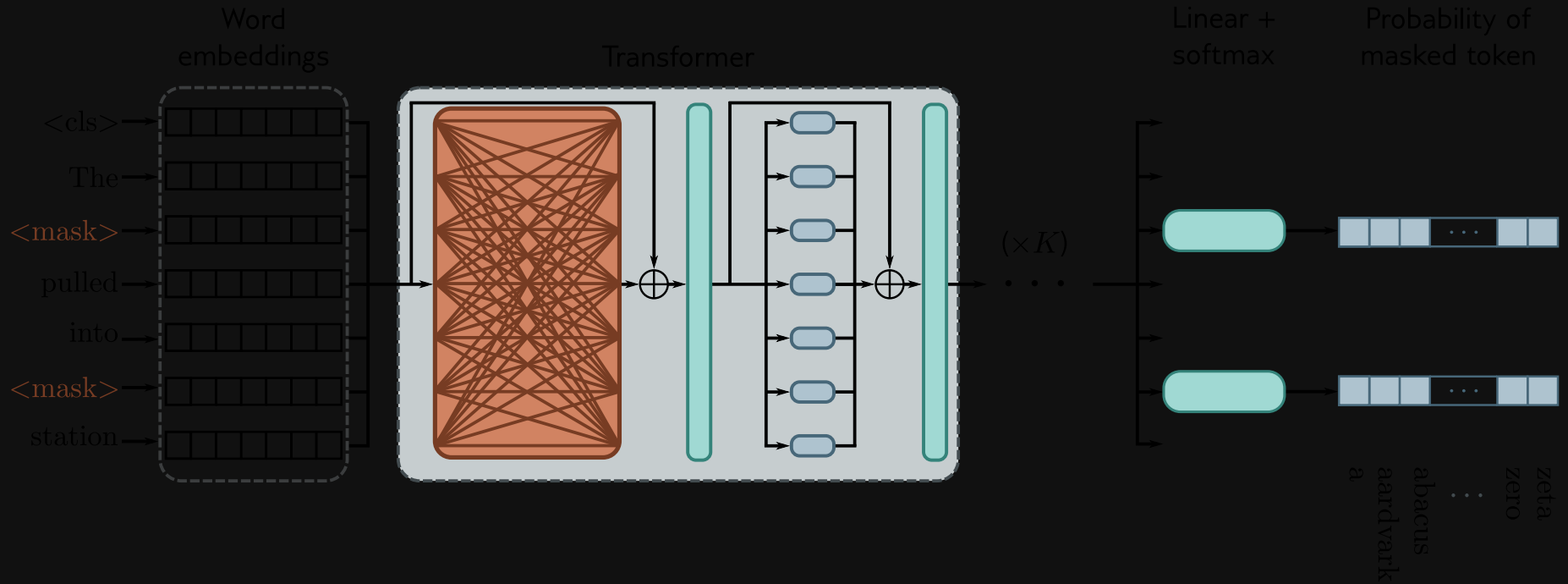
NLP Pipeline: Transformer Model

K transformer layers.

- Encoder models transform embeddings into a representation for a variety of tasks
- Decoder models are *generative* models (next lecture!): predict next token to continue input text
- Encoder-Decoder models for sequence-to-sequence tasks (e.g. translation)

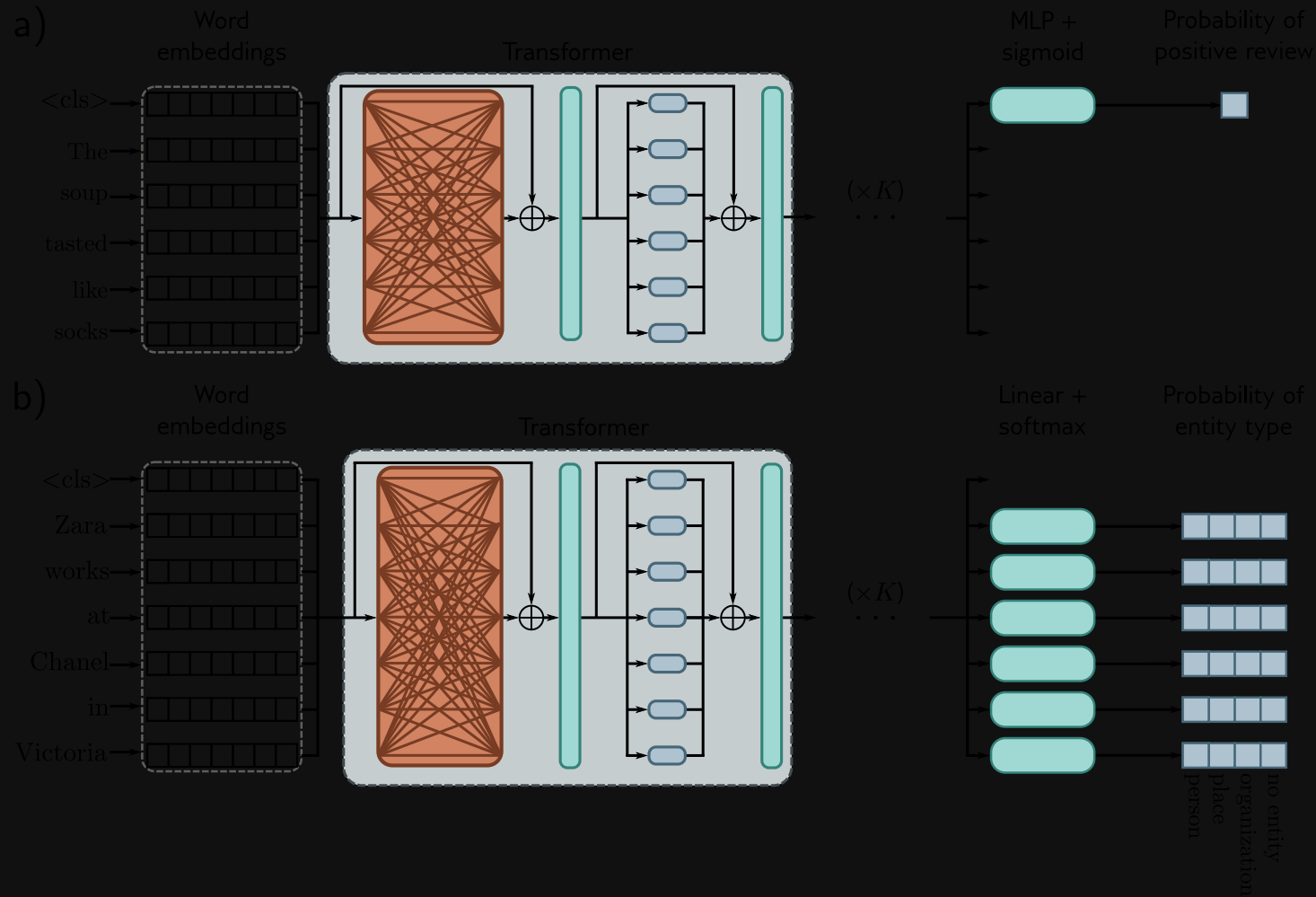
Encoder Model: BERT – Pre-Training

Vocabulary size: 30,000 tokens; 1024-dim. word embeddings; 24 transformer layers (16 heads, $D_q = 64$), 340 million parameters.



Transfer learning: pretrain on self-supervision, fine-tune on downstream tasks.

Encoder Model: BERT – Fine-Tuning



E.g. text classification, word classification, text span prediction.

References

Prince, Simon J. D. 2023. *Understanding Deep Learning*. Cambridge, Massachusetts: The MIT Press.